TASK 5 – Read Input and Convert Types

5.1 Reading Input

The input() function always returns a string.

```
litres = input("Enter litres: ")

print(type(litres))
```

Example:

```
Enter litres: 10.5

<class 'str'>
```

To perform calculations, convert the input:

```
litres = float(input("Enter litres: "))
```

For money-related inputs, use Decimal when appropriate:

```
from decimal import Decimal


litres = Decimal(input("Enter litres: "))
```

5.2 Handling Invalid Input

If the user enters "eight" instead of a number, conversion raises ValueError.

Similarly, "8,5" is not accepted by Python's ordinary float() conversion. The standard decimal separator is a period.

5.3 Input Validation Program

```
from decimal import Decimal, InvalidOperation


try:
    farmer_id = input("Enter farmer ID: ").strip()
    litres = Decimal(input("Enter litres: ").strip())
    fat = Decimal(input("Enter fat percentage: ").strip())
```

```
    snf = Decimal(input("Enter SNF percentage: ").strip())

    if not farmer_id:
        raise ValueError("Farmer ID cannot be empty.")

    if litres <= 0:
        raise ValueError("Litres must be greater than zero.")

    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0.")

    if snf < 0:
        raise ValueError("SNF cannot be negative.")

    print("Input is valid.")

except (ValueError, InvalidOperation):
    print("Invalid input. Please enter valid values.")
```

5.4 Formatted Output

```
amount = 283.90

rate = 33.40

print(f"Rate: ₹{rate:6.2f}")
```

```
print(f"Amount: ₹{amount:8.2f}")
```

Output:

```
Rate: ₹ 33.40
Amount: ₹  283.90
```

The width controls alignment, and .2f displays two decimal places.

5.5 Alternative Concept: try/except

Instead of checking the input using string methods, Python can attempt conversion and catch errors.

```
def read_decimal(prompt):
    while True:
        text = input(prompt).strip()


        try:
            return Decimal(text)
        except InvalidOperation:
            print("Invalid number. Try again.")
```

This approach accepts surrounding whitespace because of .strip(), accepts "8.5" and rejects "8,5" and "eight".

Approach	**Advantage**	**Limitation**
String checks	Can identify some formatting problems early	Does not reliably validate every numeric format
try/except	Uses Python's actual numeric conversion	Requires exception handling

Both approaches require approximately **O(1) time per input**, excluding the time needed for user interaction and the length of the input string.

Conclusion: try/except is clearer and more reliable for validating numeric input.

TASK 6 – Organise the Logic into Functions

6.1 Required Functions

The program should be divided into reusable functions.

Function	Parameters	Return value
rate_per_litre()	Fat and SNF	Rate per litre
payout()	Litres, fat and SNF	Payment amount
ten_day_total()	List of payment amounts	Total payment
validate_entry()	Input values	Validity result

6.2 Python Program

```
from decimal import Decimal


BASE_RATE = Decimal("30.00")

STANDARD_FAT = Decimal("3.5")

STANDARD_SNF = Decimal("8.5")

FAT_STEP_VALUE = Decimal("0.40")

SNF_STEP_VALUE = Decimal("0.30")

STEP_SIZE = Decimal("0.1")




def rate_per_litre(fat, snf):

    """Calculate the milk price per litre."""

    fat_steps = round((fat - STANDARD_FAT) / STEP_SIZE)
```

```
    snf_steps = round((snf - STANDARD_SNF) / STEP_SIZE)

    rate = (
        BASE_RATE
        + Decimal(fat_steps) * FAT_STEP_VALUE
        + Decimal(snf_steps) * SNF_STEP_VALUE
    )

    return rate.quantize(Decimal("0.01"))


def payout(litres, fat, snf):
    """Calculate the farmer's payment."""
    rate = rate_per_litre(fat, snf)
    return (litres * rate).quantize(Decimal("0.01"))


def ten_day_total(payments):
    """Calculate the total of individual payments."""
    return sum(payments, Decimal("0.00"))


payments = [
    payout(Decimal("10"), Decimal("3.5"), Decimal("8.5")),
```

```
    payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")),
    payout(Decimal("12"), Decimal("3.1"), Decimal("8.3"))
]

print("Ten-day total:", ten_day_total(payments))
```

Output:

```
Ten-day total: 917.50
```

Note About the Calculation

The rate calculation above uses Python's built-in round() on floating-point step calculations. For a production financial system, a more robust implementation should calculate steps from Decimal values and apply an explicitly defined tie-breaking rule.

The final design challenge below demonstrates that improved approach.

6.3 Return vs Print

return sends a result back to the caller.

```
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
```

Output:

```
30
```

print() only displays the result.

```
def add(a, b):
```

```
    print(a + b)
```

This function displays the result but returns None unless it contains a return statement.

Conclusion: Calculation functions should return values so that the results can be tested, stored and reused.

6.4 Local and Global Variables

A variable created inside a function is normally local to that function.

Constants defined outside functions can be read inside them.

For example:

```
BASE_RATE = Decimal("30.00")


def show_rate():
    """Display the standard rate."""
    return BASE_RATE
```

The function can read BASE_RATE without declaring it global.

Avoid changing global constants inside functions because it makes the program harder to understand and test.

6.5 Alternative Concept: Long Script or Lambda

A long script places all calculations in one block. It may work for a small example but becomes difficult to test and maintain.

Lambda expressions can represent short anonymous functions:

```
square = lambda x: x * x
print(square(5))
```

They are not suitable for replacing every calculation function, especially when documentation, validation and multiple steps are required.

Both modular and non-modular versions process n entries in **O(n) time**. The modular version has small function-call overhead, but its maintainability and reusability make it preferable.

TASK 7 – Build a Module

7.1 Complete the Code: milkrate.py

Save the following code as milkrate.py.

```
"""Rate card of the village milk co-operative."""


from decimal import Decimal


BASE_RATE = Decimal("30.00")

FAT_STEP_VALUE = Decimal("0.40")

SNF_STEP_VALUE = Decimal("0.30")


STANDARD_FAT = Decimal("3.5")

STANDARD_SNF = Decimal("8.5")

STEP_SIZE = Decimal("0.1")




def rate_per_litre(fat, snf):

    """Return the price per litre, rounded to two decimals."""

    fat = Decimal(str(fat))

    snf = Decimal(str(snf))


    fat_steps = round((fat - STANDARD_FAT) / STEP_SIZE)

    snf_steps = round((snf - STANDARD_SNF) / STEP_SIZE)
```

```
    rate = (
        BASE_RATE
        + Decimal(fat_steps) * FAT_STEP_VALUE
        + Decimal(snf_steps) * SNF_STEP_VALUE
    )

    return rate.quantize(Decimal("0.01"))



def payout(litres, fat, snf):
    """Return the amount payable to the farmer."""
    litres = Decimal(str(litres))
    rate = rate_per_litre(fat, snf)

    return (litres * rate).quantize(Decimal("0.01"))



if __name__ == "__main__":
    for litres, fat, snf in [
        (10, 3.5, 8.5),
        (8.5, 4.2, 8.7),
        (12, 3.1, 8.3)
    ]:
```

```
rate = rate_per_litre(fat, snf)

pay = payout(litres, fat, snf)


print(

   f"{litres:5} L  fat {fat}  snf {snf}  "

   f"rate {rate:6.2f}  pay {pay:8.2f}"

)
```

7.2 Expected Output

```
 10 L  fat 3.5  snf 8.5  rate  30.00  pay   300.00

8.5 L  fat 4.2  snf 8.7  rate  33.40  pay   283.90

 12 L  fat 3.1  snf 8.3  rate  27.80  pay   333.60
```

```
def payout(litres, fat, snf):
    """Return the amount payable to the farmer."""
    litres = Decimal(str(litres))
    rate = rate_per_litre(fat, snf)

    return (litres * rate).quantize(Decimal("0.01"))


if __name__ == "__main__":
    for litres, fat, snf in [
        (10, 3.5, 8.5),
        (8.5, 4.2, 8.7),
        (12, 3.1, 8.3)
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)

        print(
            f"{litres:5} L  fat {fat}  snf {snf}  "
            f"rate {rate:6.2f}  pay {pay:8.2f}"
        )
```

```
...    10 L  fat 3.5  snf 8.5  rate  30.00  pay   300.00
      8.5 L  fat 4.2  snf 8.7  rate  33.40  pay   283.90
       12 L  fat 3.1  snf 8.3  rate  27.80  pay   333.60
```

7.3 Importing the Module

Create a second file named test_milkrate.py.

```
import milkrate


litres = 10

fat = 3.5

snf = 8.5


rate = milkrate.rate_per_litre(fat, snf)

amount = milkrate.payout(litres, fat, snf)


print("Rate:", rate)

print("Payment:", amount)
```

Output:

```
Rate: 30.00

Payment: 300.00
```

7.4 Different Import Statements

Import the Entire Module

```
import milkrate


print(milkrate.payout(10, 3.5, 8.5))
```

Import One Function

```
from milkrate import payout
```

print(payout(10, 3.5, 8.5))

Import With an Alias

import milkrate as mr

print(mr.payout(10, 3.5, 8.5))

All three approaches allow the program to use the calculation logic.

7.5 Purpose of if __name__ == "__main__":

When a Python file is executed directly, Python sets __name__ to "__main__".

When the same file is imported into another program, __name__ is set to the module's name, such as "milkrate".

Therefore:

if __name__ == "__main__":

ensures that the demonstration table is printed only when milkrate.py is executed directly.

Without this condition, the demonstration code would run during import as well.

Python searches for modules using its import system, including locations listed in sys.path. The current project directory is normally one of these locations.

7.6 Alternative Concept: Copying Functions

If the functions were copied into every program, each copy would have to be updated whenever the rate calculation changes.

With a module, one corrected implementation can be reused by several programs.

Python normally caches imported modules in sys.modules, so repeated imports in the same interpreter generally do not execute the module again.

Importing introduces a one-time startup cost. However, this cost is usually outweighed by the benefits of reusable and maintainable code.

TASK 8 – Plan the Algorithm Before Coding

8.1 Steps of Algorithmic Problem Solving

1. **Understand:** Identify the required inputs, outputs and rules.
2. **Plan:** Develop pseudocode and a flowchart.
3. **Write:** Implement the algorithm in Python.
4. **Test:** Compare actual outputs with expected outputs and correct errors.

8.2 Pseudocode for One Payment

Algorithm: Calculate Milk Payment

```
BEGIN

    INPUT farmer_id
    INPUT litres
    INPUT fat
    INPUT snf
    INPUT shift

    IF farmer_id is empty THEN
        DISPLAY "Invalid farmer ID"
        STOP
    END IF

    IF litres <= 0 THEN
        DISPLAY "Invalid milk quantity"
```

```
    STOP
END IF

IF fat < 2.0 OR fat > 7.0 THEN
    DISPLAY "Invalid fat percentage"
    STOP
END IF

IF snf < 0 THEN
    DISPLAY "Invalid SNF percentage"
    STOP
END IF

fat_steps = ROUND((fat - 3.5) / 0.1)
snf_steps = ROUND((snf - 8.5) / 0.1)

rate = 30.00
       + fat_steps * 0.40
       + snf_steps * 0.30

rate = ROUND(rate, 2)

payment = ROUND(litres * rate, 2)
```

```
    DISPLAY farmer_id
    DISPLAY rate
    DISPLAY payment

END
```

8.3 Pseudocode for Ten-Day Total

```
BEGIN

    total = 0.00

    INPUT number_of_entries

    FOR each entry from 1 to number_of_entries

        READ farmer_id, litres, fat, snf, shift

        VALIDATE the entry

        IF the entry is valid THEN
            CALCULATE rate
            CALCULATE payment
            total = total + payment
        ELSE
```

```
        DISPLAY an error message
    END IF

END FOR

DISPLAY total

END
```

8.4 Flowchart for Validating One Entry

8.5 Hand Trace for Three Entries

Entry	Litres	Fat	SNF	Fat steps	SNF steps	Rate	Payment
1	10	3.5	8.5	0	0	₹30.00	₹300.00
2	8.5	4.2	8.7	7	2	₹33.40	₹283.90
3	12	3.1	8.3	-4	-2	₹27.80	₹333.60
Total							**₹917.50**

8.6 Alternative Concept: Store Entries or Running Total

Method 1: Store All Entries

```
entries = []

entries.append((10, 3.5, 8.5))

entries.append((8.5, 4.2, 8.7))
```

The program calculates payments later.

- Time: O(n)
- Space: O(n)

Method 2: Running Total

```
from decimal import Decimal


total = Decimal("0.00")


for litres, fat, snf in entries:
    total += payout(litres, fat, snf)
```

If entries are read one at a time and processed immediately, the program can maintain a running total without storing every record.

- Time: O(n)
- Extra space: O(1)

Conclusion: Running totals are suitable when only the total is required. Storing records is necessary when the co-operative needs to audit transactions, generate reports or recalculate payments.